



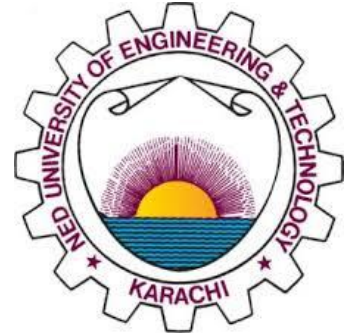
ARCADE HUB PRO

C# GUI Application

By :- Hadiya Ayesah (SE-25016)

ARCADE HUB PRO PROJECT REPORT

(Complex Engineering Problem -CEP)



Element	Content
Title	ARCADE HUB PRO PROJECT
Subtitle	A C# GUI Application
Submitted By:	Hadiya Ayesah (SE-25016)
Course Name	Object-Oriented Programming
Course Code	SE-201
Submitted To:	Miss Asma
Semester	Spring-2026
Date	11 May 2026

1. Problem Statement

Modern users enjoy arcade-style mini games that provide entertainment, challenge, and interactive gameplay within a single application. However, many simple game applications lack proper visual design, smooth navigation, responsive interfaces, and integrated gameplay systems. Managing multiple games inside one desktop application also introduces challenges such as state management, timer handling, score tracking, and smooth transitions between game modules.

This project addresses these challenges by developing Arcade Hub Pro, a GUI-based arcade application using C# Windows Forms. The system combines multiple mini-games into a single modern interface, including Word Search, Memory Match, and Balloon Pop. The application manages game switching, scoring systems, timers, animations, and interactive gameplay while maintaining a visually appealing neon-themed interface.

The project emphasizes modular programming, event-driven design, graphical user interface development, and reusable components. The application also demonstrates dynamic layouts, animation effects, collision handling, timers, and responsive user interaction. The main requirements were:

- Create a modern GUI-based arcade application using C# Windows Forms.
- Develop multiple integrated mini-games inside one system.
- Implement score tracking and countdown timers.
- Design animated and visually appealing interfaces.
- Create responsive buttons, hover effects, and transitions.
- Maintain modular and reusable code structure.
- Provide smooth navigation between games.
- Display mission completion and final game status.

2. Table of Contents:-

1.	Title Page
2.	Introduction
3.	Problem Statement ³
4.	Table of Contents ⁴
5.	Project Objective⁵
6.	Methodology⁵
5.	Distinguishing Features ⁸
6.	Screenshots Of Main Features⁹
7.	Key Code Snippets ¹²
8.	Future Expansions ¹⁵
9.	Conclusion ¹⁵

3. Project Objective

The objective of this project is to design and implement a fully interactive arcade gaming application using C# and Windows Forms. The application aims to combine multiple mini-games inside one unified environment while maintaining smooth gameplay, attractive graphics, and user-friendly controls.

The project focuses on creating an engaging experience through neon-themed visuals, animations, dynamic game transitions, and responsive controls. Each game module demonstrates different programming concepts including grid-based logic, event handling, timers, randomization, and user interaction.

Another objective is to implement a modular software architecture that allows future expansion and easy maintenance. The project also aims to improve practical knowledge of GUI design, object-oriented programming, event-driven systems, and desktop application development. Ultimately, the application delivers a complete arcade experience with score systems, game completion tracking, and visually polished gameplay.

4. Methodology

The development of Arcade Hub Pro followed a structured and modular methodology to ensure smooth functionality and maintainable code.

4.1. Requirements Analysis

The project requirements were identified by analyzing the features commonly present in arcade-style gaming systems. Key requirements included:

- Multiple integrated mini-games
- Score tracking system
- Countdown timer
- Animated graphical interface
- Smooth navigation between games

4.2. System Design

The application was divided into different modules:

- FrontPage Module: Handles title screen and game selection.
- Theme Module: Manages colors, graphics, borders, and UI styling.
- Word Search Module: Controls grid generation and word detection.
- Memory Match Module: Handles card flipping and pair matching.
- Balloon Pop Module: Controls balloon movement and popping logic.
- Statistics Module: Tracks timer, score, and progress.

4.3. GUI Development

Windows Forms was used to build the graphical interface. Major GUI tasks included:

- Designing responsive layouts
- Creating custom neon-style themes
- Adding animated hover effects
- Implementing custom drawing using Graphics and GDI+
- Designing reusable buttons and panels

5: Game Logic Implementation

Each game implemented with complete rule set and state management:

Word Search

Procedural grid generation with word placement algorithm, adjacent cell validation, and word tracking visualization.

Memory Match

8-card pair matching with flip animation, match detection, and disable logic for completed pairs.

Balloon Pop

10 animated balloons with vertical floating motion, click-to-pop detection, and progress bar visualization.

6. Testing and Debugging

The application was tested repeatedly to ensure:

- Correct score calculations
- Smooth transitions between games
- Accurate word detection
- Proper memory matching behavior
- Stable balloon movement and collision handling

7. Refinement and Optimization

The final phase included:

- Improving UI responsiveness
- Enhancing animations
- Optimizing timers and rendering
- Improving overall visual presentation
- Organizing reusable code modules

8. Distinguishing Features of the Project

8.1. Modern Neon-Themed Interface

The application uses custom colors, glowing borders, gradients, and animated effects to create a futuristic arcade experience.

8.2. Multiple Mini-Games

The system integrates three different games:

- Word Search
- Memory Match
- Balloon Pop

8.3. Real-Time Score and Timer System

Players can monitor:

- Remaining time
- Current score
- Game progress

8.4. Responsive GUI Design

The interface adjusts dynamically and uses smooth layouts with custom graphics and animations.

8.5. Interactive Gameplay

The games include:

- Click-based interactions
- Hover effects
- Dynamic transitions
- Real-time updates

8.6. Modular Code Structure

The project uses reusable classes and organized modules, making future expansion easier.

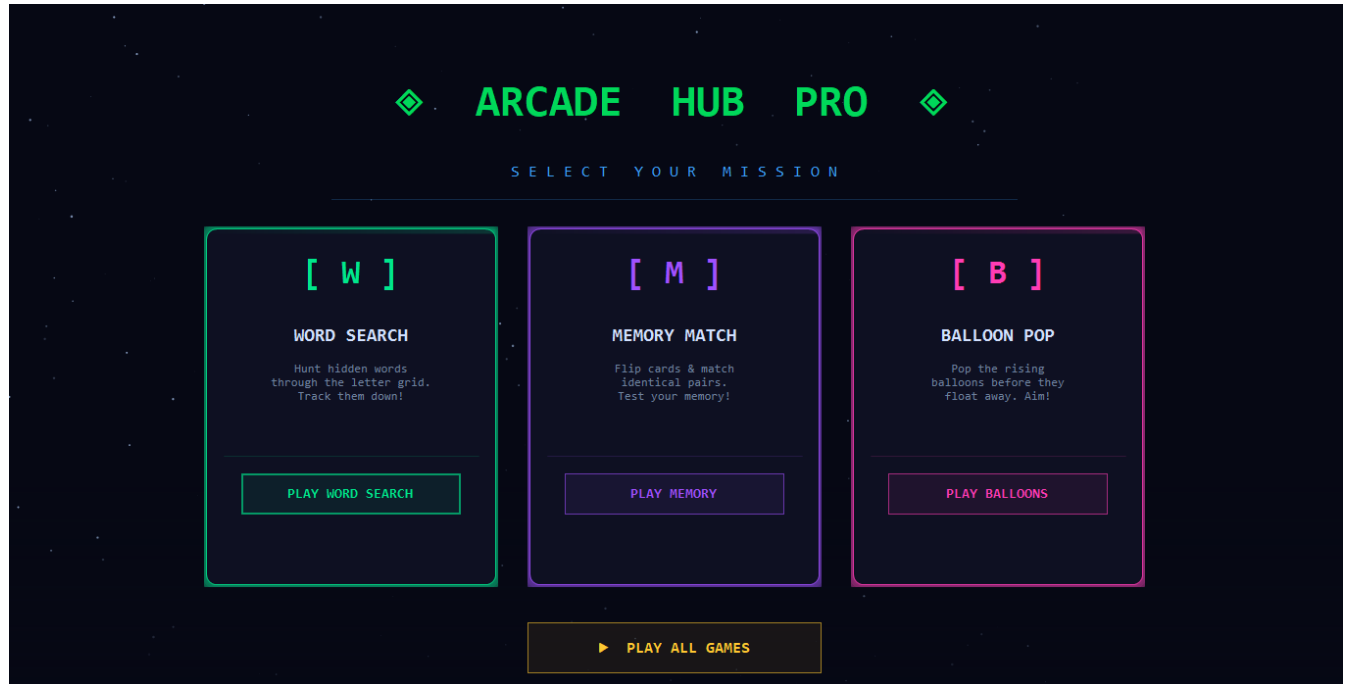
8.7. Animated Visual Effects

The system includes:

- Floating background stars
- Neon glow borders
- Animated buttons
- Balloon movement animations

9. Screenshots of Main Features

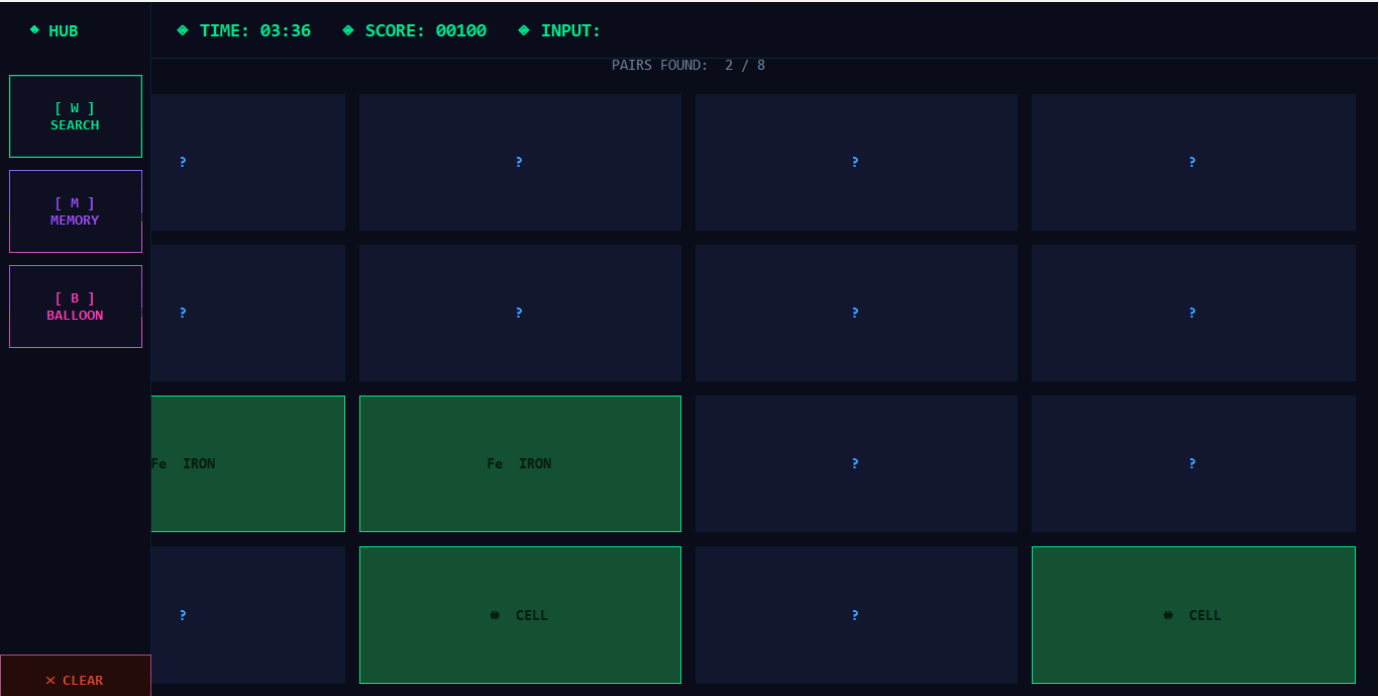
1. Front Page / Title Screen



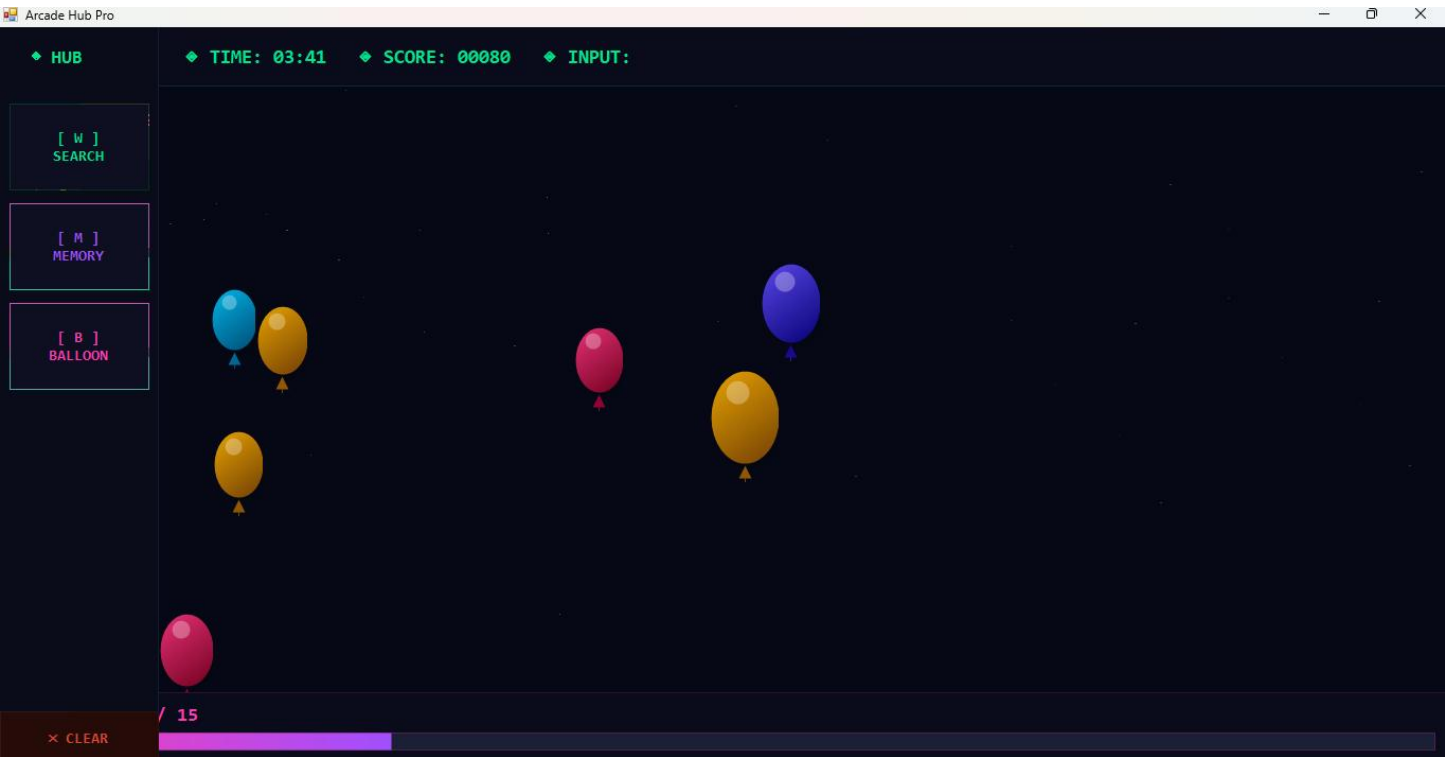
2. Word Search Game



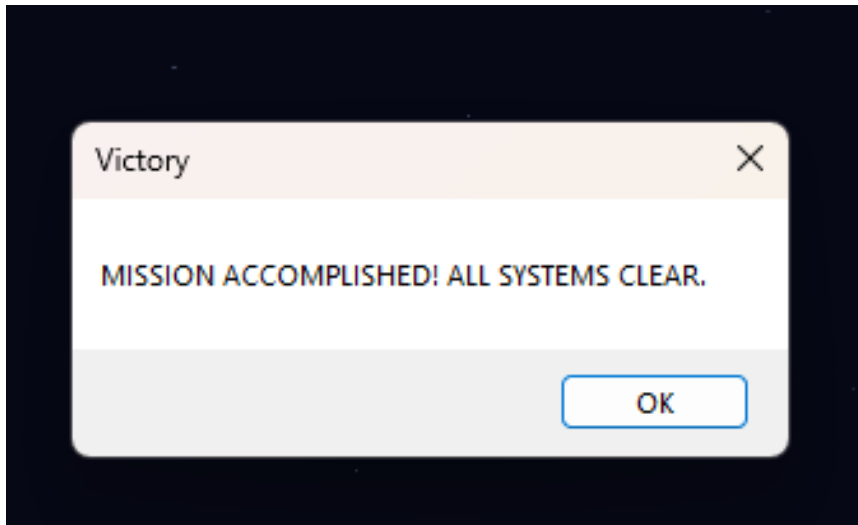
3. Memory Match Game



4. Balloon Pop Game



5. Mission Complete Screen



10. Source Code of Main Features

I. Theme and Graphics System

```
public FrontPage()
{
    this.Text = "ARCADE HUB PRO";
    this.Size = new Size(1100, 800);
    this.StartPosition = FormStartPosition.CenterScreen;
    this.BackColor = Theme.Bg;
    this.DoubleBuffered = true;
    this.FormBorderStyle = FormBorderStyle.Sizable;
    this.MaximizeBox = true;
}

67 references
static class Theme
{
    public static readonly Color Bg = Color.FromArgb(6, 8, 20);
    public static readonly Color BgCard = Color.FromArgb(14, 16, 34);
    public static readonly Color BgPanel = Color.FromArgb(10, 12, 26);
    public static readonly Color Neon = Color.FromArgb(0, 230, 140);
    public static readonly Color Blue = Color.FromArgb(60, 160, 255);
    public static readonly Color Purple = Color.FromArgb(160, 80, 255);
    public static readonly Color Pink = Color.FromArgb(255, 60, 180);
    public static readonly Color Gold = Color.FromArgb(255, 200, 50);
    public static readonly Color TextDim = Color.FromArgb(110, 130, 160);
    public static readonly Color TextBright = Color.FromArgb(210, 225, 255);
}
```

II. Front Page Design

```
var btnAll = MakeNeonButton("▶ PLAY ALL GAMES", Theme.Gold);
btnAll.Top = 632; btnAll.Width = 300; btnAll.Height = 52;
btnAll.Left = (1100 - 300) / 2;
btnAll.Font = new Font("Consolas", 12, FontStyle.Bold);
btnAll.Anchor = AnchorStyles.Top;
btnAll.Click += (s, e) => LaunchGame(0);
this.Controls.Add(btnAll);
this.Resize += (s, e) => { btnAll.Left = (this.ClientSize.Width - btnAll.Width) / 2; };

string[] names = { "WORD SEARCH", "MEMORY MATCH", "BALLOON POP" };
string[] icons = { "[ W ]", "[ M ]", "[ B ]" };
string[] descs = {
    "Hunt hidden words\nthrough the letter grid.\nTrack them down!",
    "Flip cards & match\nidentical pairs.\nTest your memory!",
    "Pop the rising\nballoons before they\nfloat away. Aim!"
};
string[] btns = { "PLAY WORD SEARCH", "PLAY MEMORY", "PLAY BALLOONS" };
Color[] accents = { Theme.Neon, Theme.Purple, Theme.Pink };

var lbSub = new Label
{
    Text = "S E L E C T   Y O U R   M I S S I O N",
    Font = new Font("Consolas", 12),
    ForeColor = Theme.Blue,
    BackColor = Color.Transparent,
    AutoSize = false,
    Anchor = AnchorStyles.Top | AnchorStyles.Left | AnchorStyles.Right,
    Left = 0,
    Top = 155,
    Width = 1100,
    Height = 32,
    TextAlign = ContentAlignment.MiddleCenter
};
this.Controls.Add(lbSub);
this.Resize += (s, e) => { lbSub.Width = this.ClientSize.Width; };
```

```

private void BuildUI()
{
    lblTitle = new Label
    {
        Text = "◊ ARCADE HUB PRO ◊",
        Font = new Font("Consolas", 34, FontStyle.Bold),
        ForeColor = Theme.Neon,
        BackColor = Color.Transparent,
        AutoSize = false,
        Dock = DockStyle.None,
        Anchor = AnchorStyles.Top | AnchorStyles.Left | AnchorStyles.Right,
        Left = 0,
        Top = 55,
        Width = 1100,
        Height = 90,
        TextAlign = ContentAlignment.MiddleCenter
    };
    this.Controls.Add(lblTitle);
    this.Resize += (s, e) => { if (lblTitle != null) lblTitle.Width = this.ClientSize.Width; };

    pnlNav = new Panel
    {
        Dock = DockStyle.Left,
        Width = 150,
        BackColor = Color.FromArgb(10, 12, 26)
    };
    pnlNav.Paint += (s, e) => {
        using (var pen = new Pen(Color.FromArgb(30, Theme.Blue), 1))
            e.Graphics.DrawLine(pen, pnlNav.Width - 1, 0, pnlNav.Width - 1, pnlNav.Height);
        using (var f = new Font("Consolas", 13, FontStyle.Bold))
        using (var br = new SolidBrush(Theme.Neon))
            e.Graphics.DrawString("◊ HUB", f, br, 26, 18);
    };

    string[] menu = { "SEARCH", "MEMORY", "BALLOON" };
    Color[] mColors = { Theme.Neon, Theme.Purple, Theme.Pink };
    string[] mIcons = { "[ W ]", "[ M ]", "[ B ]" };

    ,
    topBar.Paint += (s, e) => {
        using (var pen = new Pen(Color.FromArgb(40, Theme.Neon), 1))
            e.Graphics.DrawLine(pen, 0, topBar.Height - 1, topBar.Width, topBar.Height - 1);
    };
    lblStats.Dock = DockStyle.Fill;
    lblStats.ForeColor = Theme.Neon;
    lblStats.Font = new Font("Consolas", 14, FontStyle.Bold);
    lblStats.TextAlign = ContentAlignment.MiddleLeft;
    lblStats.BackColor = Color.Transparent;
    topBar.Controls.Add(lblStats);
    this.Controls.Add(topBar);

    pnlNav = new Panel
    {
        Dock = DockStyle.Left,
        Width = 150,
        BackColor = Color.FromArgb(10, 12, 26)
    }

```

III. Word Search Logic

```
1 reference
private void LoadWordSearch()
{
    if (wordSearchDone) { ShowDone("WORD SEARCH", "All words found!", Theme.Neon); return; }

    var allWords = new List<string> { "SPACE", "ATOM", "STARS", "MOON", "CELL" };
    if (wordEntries.Count == 0)
        foreach (var w in allWords)
            wordEntries.Add(new WordEntry { Word = w, Found = false });

    RebuildWordBank();

    char[,] map = new char[10, 10];
    var rnd = new Random();
    (int dr, int dc)[] dirs = { (0, 1), (1, 0) };

    foreach (var entry in wordEntries)
    {
        if (entry.Found) continue;
        string word = entry.Word;
        int L = word.Length;
        bool placed = false;
        var shuffledDirs = dirs.OrderBy(_ => rnd.Next()).ToArray();

        private void RebuildWordBank()
        {
            foreach (var l in wbLabels) pnlWordBank.Controls.Remove(l);
            wbLabels.Clear();
            var all = new List<string> { "SPACE", "ATOM", "STARS", "MOON", "CELL" };
            for (int i = 0; i < all.Count; i++)
            {
                bool found = !targets.Contains(all[i]);
                var lbl = new Label
                {
                    Text = (found ? "✓" : "»") + all[i],
                    Font = new Font("Consolas", 11, FontStyle.Bold),
                    ForeColor = found ? Color.FromArgb(60, Theme.Neon) : Theme.Neon,
                    BackColor = Color.Transparent,
                    AutoSize = false,
                    Width = 178,
                    Height = 32,
                    Top = 46 + i * 38,
                    TextAlign = ContentAlignment.MiddleLeft,
                    Padding = new Padding(10, 0, 0, 0)
                };
                pnlWordBank.Controls.Add(lbl);
                wbLabels.Add(lbl);
            }
        }

        private void placeWord()
        {
            for (int row = 0; row < 10 && !placed; row++)
            {
                for (int col = 0; col <= 10 - L && !placed; col++)
                {
                    bool ok = true;
                    for (int i = 0; i < L && ok; i++)
                    {
                        char ex = map[row, col + i];
                        if (ex != '\0' && ex != word[i]) ok = false;
                    }
                    if (ok)
                    {
                        entry.Cells.Clear();
                        for (int i = 0; i < L; i++)
                        {
                            map[row, col + i] = word[i];
                            entry.Cells.Add((row, col + i));
                        }
                        placed = true;
                    }
                }
            }
        }
    }
}
```

```

for (int attempt = 0; attempt < 800 && !placed; attempt++)
{
    var (dr, dc) = shuffledDirs[attempt % shuffledDirs.Length];
    int maxStartR = 9 - dr * (L - 1);
    int maxStartC = 9 - dc * (L - 1);
    if (maxStartR < 0 || maxStartC < 0) continue;
    int startR = rnd.Next(0, maxStartR + 1);
    int startC = rnd.Next(0, maxStartC + 1);
    bool ok = true;
    for (int i = 0; i < L && ok; i++)
    {
        char ex = map[startR + dr * i, startC + dc * i];
        if (ex != '\0' && ex != word[i]) ok = false;
    }
    if (!ok) continue;
    entry.Cells.Clear();
    for (int i = 0; i < L; i++)
    {
        int cr = startR + dr * i, cc = startC + dc * i;
        map[cr, cc] = word[i];
        entry.Cells.Add((cr, cc));
    }
}

```

IV. Memory Match Logic

```

string[] words = { "ATOM", "DNA", "BOND", "CELL", "STAR", "IRON", "O2", "GOLD" };
string[] syms = { "☼", "DNA", "☼", "e", "★", "Fe", "O²", "AU" };
Color[] accents = {
    Theme.Neon, Theme.Blue, Theme.Purple, Theme.Pink,
    Theme.Gold, Color.FromArgb(100, 200, 255), Color.FromArgb(255, 160, 60), Color.FromArgb(60, 220,
};
var accentMap = new Dictionary<string, Color>();
var symMap = new Dictionary<string, string>();
for (int i = 0; i < words.Length; i++) { accentMap[words[i]] = accents[i]; symMap[words[i]] = sy
var gameSet = new List<string>(words);
gameSet.AddRange(words);
var shuffled = gameSet.OrderBy(_ => Guid.NewGuid()).ToList();

// lblPairs reference needed inside card click handler
var lblPairs = new Label
{
    Text = "PAIRS FOUND: 0 / 8",
    Font = new Font("Consolas", 11),
    ForeColor = Theme.TextDim,
    BackColor = Color.Transparent,
    Dock = DockStyle.Top,
    Height = 28.

```

```

for (int i = 0; i < 16; i++)
{
    string word = shuffled[i];
    Color accent = accentMap[word];
    string sym = symMap[word];

    var card = new Button
    {
        Dock = DockStyle.Fill,
        FlatStyle = FlatStyle.Flat,
        BackColor = Color.FromArgb(18, 22, 46),
        ForeColor = Theme.Blue,
        Font = new Font("Consolas", 11, FontStyle.Bold),
        Tag = word,
        Text = "?",
        Margin = new Padding(7),
        Cursor = Cursors.Hand
    };
    card.FlatAppearance.BorderColor = Color.FromArgb(45, Theme.Blue);
    card.FlatAppearance.BorderSize = 1;

    card.MouseEnter += (s, e) => {
        if (card.Text == "?" && card.Enabled)
            card.BackColor = Color.FromArgb(28, 32, 62);
    };
    card.MouseLeave += (s, e) => {
        if (card.Text == "?" && card.Enabled)
            card.BackColor = Color.FromArgb(18, 22, 46);
    };
}

```

```

card.Click += (s, e) => {
    if (card.Text != "?" || !card.Enabled) return;

    card.Text = sym + " " + word;
    card.BackColor = Color.FromArgb(28, accent.R, accent.G, accent.B);
    card.ForeColor = accent;
    card.FlatAppearance.BorderColor = accent;

    if (memFirst == null)
    {
        memFirst = card;
    }
    else
    {
        pnlGame.Enabled = false;
        if (memFirst.Tag.ToString() == card.Tag.ToString() && memFirst != card)
        {
            score += 50; memoryMatches++;
            card.Enabled = memFirst.Enabled = false;
            card.BackColor = memFirst.BackColor = Color.FromArgb(20, 80, 50);
            card.ForeColor = memFirst.ForeColor = Theme.Neon;
            card.FlatAppearance.BorderColor = Theme.Neon;
            memFirst.FlatAppearance.BorderColor = Theme.Neon;
            memFirst = null;
            pnlGame.Enabled = true;
            lblPairs.Text = string.Format("PAIRS FOUND: {0} / 8", memoryMatches);
            if (memoryMatches == 8)
            {
                ;
            }
        }
        else
        {
            var firstRef = memFirst; memFirst = null;
            var flip = new Timer { Interval = 800 };
            flip.Tick += (st, se) => {
                flip.Stop();
                foreach (var cr in new[] { card, firstRef })
                {
                    cr.Text = "?";
                    cr.BackColor = Color.FromArgb(18, 22, 46);
                    cr.ForeColor = Theme.Blue;
                    cr.FlatAppearance.BorderColor = Color.FromArgb(45, Theme.Blue);
                }
                pnlGame.Enabled = true;
            };
            flip.Start();
        }
    }
}

```

V. Balloon Pop Logic

```

private void LoadBalloons()
{
    pnlGame.Controls.Clear();
    if (balloonsDone) { ShowDone("BALLOON POP", "All balloons popped!", Theme.Pink); return; }

    var area = new Panel { Dock = DockStyle.Fill, BackColor = Color.FromArgb(6, 8, 22) };

    area.Paint += (s, e) => {
        var g = e.Graphics;
        var rr = new Random(77);
        for (int i = 0; i < 70; i++)
        {
            int x = rr.Next(area.Width), y = rr.Next(area.Height - 70);
            using (var br = new SolidBrush(Color.FromArgb(rr.Next(30, 110), 180, 210, 255)))
            {
                g.FillEllipse(br, x, y, rr.Next(1, 3), rr.Next(1, 3));
            }
        }
    };

    area.Controls.Add(new Label
    {
        Text = "♦ B A L L O O N   P O P ♦",
        Font = new Font("Consolas", 15, FontStyle.Bold),
        ForeColor = Theme.Pink,
        BackColor = Color.Transparent,
        Dock = DockStyle.Top,
        Height = 48,
        TextAlign = ContentAlignment.MiddleCenter
    });
}

```

```

        Text = string.Format(" ♦ POPPED: {0} / 15", balloonsPopped),
        Font = new Font("Consolas", 13, FontStyle.Bold),
        ForeColor = Theme.Pink,
        BackColor = Color.Transparent,
        AutoSize = false,
        Width = 400,
        Height = 30,
        Top = 6,
        TextAlign = ContentAlignment.MiddleLeft
    };
    bottomBar.Controls.Add(lblC);
    bottomBar.Paint += (s, e) => {
        var g = e.Graphics;
        g.SmoothingMode = SmoothingMode.AntiAlias;
        using (var pen = new Pen(Color.FromArgb(40, Theme.Pink), 1))
            g.DrawLine(pen, 0, 0, bottomBar.Width, 0);
        int barW = bottomBar.Width - 24;
        int fillW = (int)(balloonsPopped / 15.0 * barW);
        using (var br = new SolidBrush(Color.FromArgb(28, 32, 55)))
            g.FillRectangle(br, 12, 38, barW, 16);
        if (fillW > 0)
            using (var lgb = new LinearGradientBrush(
                new Point(12, 38), new Point(12 + fillW, 38),
                Theme.Pink, Theme.Purple))
                g.FillRectangle(lgb, 12, 38, fillW, 16);
        using (var pen = new Pen(Color.FromArgb(70, Theme.Pink), 1))
            g.DrawRectangle(pen, 12, 38, barW, 16);
    };
    for (int i = 0; i < 10; i++)
    {
        int bw = rnd.Next(44, 68);
        int bh = (int)(bw * 1.35);
        int speed = rnd.Next(3, 9);
        Color col = bColors[rnd.Next(bColors.Length)];
        int bww = bw, bhh = bh;
        Color cc = col;

        var b = new Button
        {
            Size = new Size(bw, bh + 18),
            FlatStyle = FlatStyle.Flat,
            BackColor = Color.Transparent,
            Cursor = Cursors.Hand
        };
        b.FlatAppearance.BorderSize = 0;
        b.FlatAppearance.MouseOverBackColor = Color.Transparent;
        b.FlatAppearance.MouseDownBackColor = Color.Transparent;
        b.Location = new Point(rnd.Next(30, 720), rnd.Next(520, 920));

        b.Paint += (s, e) => {
            var g = e.Graphics;
            g.SmoothingMode = SmoothingMode.AntiAlias;
            using (var br = new SolidBrush(Color.FromArgb(40, 0, 0)))
                g.FillEllipse(br, 5, 5, bww, bhh);
        };

        var t = new Timer { Interval = 22 };
        t.Tick += (s, e) => { b.Top -= speed; if (b.Bottom < 50) b.Top = area.Height - 80; };

        b.MouseDown += (s, e) => {
            score += 20; balloonsPopped++;
            lblC.Text = string.Format(" ♦ POPPED: {0} / 15", balloonsPopped);
            bottomBar.Invalidate();
            b.Top = area.Height + 200;
            if (balloonsPopped >= 15) { t.Stop(); balloonsDone = true; RefreshStats(); SwitchGame(2);
        };
    }
}

```


11. Game Mechanics

11.1 : Word Search

Objective

Find all five target words (SPACE, ATOM, STARS, MOON, CELL) in a 10×10 letter grid by selecting adjacent letters in a straight line.

Implementation

- Procedural grid generation places words horizontally or vertically
- Button array (10×10) maps to char array positions
- OnWordCellClick validates adjacency and direction consistency
- Word matching checks selection against target entry
- Found words disabled (grayed out) and marked in word bank

Scoring

100 points per word found

11.2 : Memory Match

Objective

Match all 8 pairs of cards by flipping them to reveal symbols and text.

Implementation

- 4×4 TableLayoutPanel with 16 buttons
- Cards initialized as '?' (facedown)
- Click handler reveals symbol + word and stores reference in memFirst
- Second card comparison: Tag equality determines match success
- Mismatch triggers 800ms flip timer returning cards to '?' state
- Match disables both cards and increments memoryMatches counter

Scoring

50 points per matched pair

11.3 Balloon Pop

Objective

Pop all 15 balloons before they float off the top of the screen.

Implementation

- 10 button instances with custom Paint handlers
- Timer.Tick updates position (Top -= speed)
- Wrapping: b.Top = area.Height - 80 when balloon exits top
- MouseDown handler increments balloonsPopped and moves balloon off-screen

Scoring

20 points per balloon popped

12. Future Expansions

The project can be expanded with several advanced features:

- **Sound Effects and Background Music**

Add immersive audio feedback for gameplay events.

- **Additional Mini-Games**

Introduce racing, shooting, puzzle, or reaction-based games.

- **Multiplayer Support**

Allow players to compete together online or locally.

- **Advanced Animations**

Implement smoother transitions and particle effects.

- **High Score System**

Store player records and rankings.

- **User Profiles and Save System**

Allow players to save progress and customize settings.

- **Difficulty Levels**

Add beginner, intermediate, and advanced gameplay modes.

13. Conclusion

The Arcade Hub Pro project successfully demonstrates the development of a modern multi-game desktop application using C# Windows Forms. The application combines multiple interactive mini-games into a single visually engaging arcade environment. The project effectively implements graphical user interface design, event-driven programming, timers, animations, and modular game logic. Features such as neon themes, score systems, responsive layouts, and animated gameplay contribute to an enjoyable user experience. The project also strengthens programming concepts including object-oriented programming, GUI development, graphics rendering, and logical problem solving. The modular structure of the application makes it suitable for future expansion and feature enhancement. Overall, the project achieves its objective of delivering a polished, interactive, and entertaining arcade gaming system while providing practical experience in desktop application development and game design.